# *Interactive Living Room: A Dynamic Computer Graphics Scene*

**Submitted By**


| Student Name | Student ID |
|---|---|
| Shipon Chandra Mondal | 0242220005101741 |
| Romana Akter Runa | 0242220005101597 |
| Ismail Hoissen | 0242220005101797 |


**LAB PROJECT REPORT**

This Report Presented in Partial Fulfillment of the course

**CSE422: Computer Graphics Lab in the Department of Computer Science and Engineering**


**DAFFODIL INTERNATIONAL UNIVERSITY**

**Dhaka, Bangladesh**


**19/04/2026**

# DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **Most. Sanjida Afrin**, **Lecturer**, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

**Submitted To:**

**Most. Sanjida Afrin**

_____

**Most. Sanjida Afrin**
**Lecturer,**
Department of Computer Science and Engineering
Daffodil International University

**Submitted by**

| | |
|---|---|
| **Shipon Chandra Mondal**<br>Student Name Student<br>ID:0242220005101741<br>Dept. of CSE, DIU | |
| **Romana Akter Runa**<br>Student Name Student<br>ID:0242220005101597<br>Dept. of CSE, DIU | **Ismail Hoissen**<br>Student Name Student<br>ID:0242220005101797<br>Dept. of CSE, DIU |

# COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:

Table 1: Course Outcome Statements

| | |
|---|---|
| **CO1** | Design and implement fundamental computer graphics primitives and algorithms to construct 2D graphical scenes for real-life visualization problems having complex engineering attribute |
| **CO2** | Apply appropriate OpenGL programming techniques, geometric transformations, and graphics algorithms to solve complex real-world visualization problems using modern engineering tools. |
| **CO3** | Perform effectively as an individual and as a member of a diverse team to design, document, and implement computer graphics–based projects. |
| **CO4** | Create and present a complete computer graphics project by explaining complex engineering activities through effective reports, demonstrations, and presentations. |

## Mapping Course Outcome (COs) with the Teaching-Learning and Assessment Strategy

The mapping justification of this table is provided in section **4.2.1**, **4.2.2** and **4.2.3**.

Table 2: Mapping of CO, PO, Blooms, KP and CEP

| **CO** | **POs** | **Teaching Learning Activities** | **Assessment Strategy** | **Learning Domains** | **Knowledge Profile** | **Complex Engineering Problem** | **Complex Engineering Activities** |
|---|---|---|---|---|---|---|---|
| CO1 | PO1 | TLA1 (Lab exercises) | Lab performance, Lab report | C2 | K1, K3 | EP1,EP4 | |
| CO2 | PO5 | TLA2 (Algorithm implementation) | Lab performance, Lab report | C3, C4 | K6 | EP1, EP4, EP3 | |
| CO3 | PO9 | TLA3 (Group project) | Lab project | P3, A2 | | | |
| CO4 | PO10 | TLA4 (Presentation & Viva) | Project & Viva | C6, P3, A2 | | | EA1, EA3 |

# Table of Contents


**Declaration** **i**

**Course & Program Outcome** **ii**

**1 Introduction** **1**
1.1 Introduction 1
1.2 Motivation 1
1.3 Objectives 1
1.4 Feasibility Study 1
1.5 Gap Analysis 1
1.6 Project Outcome 1

**2 System Architecture / Scene Description & Architecture** **2**
2.1 Requirement Analysis & Design Specification 2
2.1.1 Requirements 2
2.1.2 Sketch of project 2
2.2 Use of Modern Tools 2
2.3 Algorithm Used **2**
2.4 2D Transformations Applied **2**
2.5 Animation Logic 2

**3 Implementation and Results** **3**
3.1 Implementation 3
3.2 Output 3
3.3 Discussion 3

**4 Engineering Standards and Mapping** **4**
4.1 Impact on Society, Environment and Sustainability 4
4.1.1 Impact on Life 4
4.1.2 Impact on Society & Environment 4
4.1.3 Ethical Aspects 4
4.1.4 Sustainability Plan 4
4.2 Complex Engineering Problem 4
4.2.1 Mapping of Program Outcome 4
4.2.2 Complex Problem Solving 4
4.2.3 Engineering Activities 5

**5 Conclusion** **6**
5.1 Summary 6
5.2 Limitation 6
5.3 Future Work 6

**References** **6**

# Chapter 1

# Introduction

This chapter provides an overview of the Computer Graphics project, including its objectives, scope, and significance. It establishes the foundation by explaining the project's theme, visual concept, and the goals it aims to achieve through graphical modeling and animation.

## 1.1 Introduction

Computer graphics is a powerful field that enables the creation of visual content using computational methods. This project, "Interactive Living Room," utilizes the OpenGL library and C++ to develop an interactive 2D scene representing a modern living room. The project goes beyond static drawing by incorporating real-time animations, dynamic lighting effects, and user interaction. It serves as a practical application of fundamental graphics primitives, algorithms (Midpoint Circle), and 2D geometric transformations to build a cohesive and engaging virtual environment.

## 1.2 Motivation

Our team was motivated to create this project to bridge the gap between theoretical computer graphics concepts and their practical implementation in a creative context. We wanted to build an interactive system that is both visually appealing and technically robust, showcasing how algorithms and transformations can bring a static design to life. The choice of a living room scene provides a familiar and relatable theme, allowing us to focus on complex details like realistic lighting, smooth animation of a cat's tail, and a dynamic clock, all while maintaining a user-friendly interactive experience.

## 1.3 Objectives

- To implement the Midpoint Circle algorithm for drawing curved objects like the clock, lamp glow, and moving ball.
- To apply 2D transformations (translation for object movement, rotation for the clock's second hand and cat's tail).
- To create an animated graphical scene with multiple moving elements (cat's tail, clock seconds hand, glowing effects).
- To implement real-time user interaction for toggling a dynamic lighting system and controlling an object on the TV screen.
- To demonstrate modular programming principles by creating dedicated functions for each graphical component.

## 1.4 Feasibility Study

A feasibility study was conducted by reviewing similar OpenGL-based projects available on platforms like GitHub and educational resources like GeeksforGeeks [2]. Existing projects often demonstrate static scenes or simple animations. For example, many "Village Scenery" projects show moving clouds or a sun but lack complex, state-based interactions like lighting that affects the entire scene's color palette. Case studies on interactive applications reveal that real-time rendering and user input are technically challenging but achievable with OpenGL's efficient callback system. This analysis confirmed that our project, with its focus on dynamic lighting and multi-object animation, represents a methodological contribution by integrating several complex graphics techniques into a single, interactive application.

## 1.5 Gap Analysis

While many computer graphics lab projects focus on demonstrating individual algorithms (e.g., drawing a house with DDA lines) or simple animations (e.g., a bouncing ball), they often fail to integrate these elements into a cohesive, interactive system. Key gaps identified include: a lack of dynamic, state-based lighting that affects visual perception; limited use of concurrent animations (e.g., a rotating clock hand, a wagging tail, and a moving object); and minimal user interaction beyond basic object control. Our project intends to fill these gaps by creating a fully integrated scene where the lamp's state alters the scene's ambiance, multiple independent animations run simultaneously, and the user can actively engage with the environment.

## 1.6 Project Outcome

The expected outcome of this work is a fully functional, interactive 2D living room scene rendered with OpenGL. The final output will feature a realistic room with a sofa, a beautifully detailed cat with an animated tail, a functional wall clock, a TV unit, and a lamp that toggles a global lighting effect. Users will be able to control a moving ball on the TV screen using arrow keys. The project will successfully demonstrate the application of computer graphics algorithms, 2D transformations, and real-time animation techniques as outlined in the project's objectives.

# Chapter 2

# System Architecture/ Scene Description & Architecture

This chapter describes the overall scene design and architecture of the Computer Graphics project. It outlines the main graphical components, their interactions, and the technologies used to create an efficient, visually appealing, and animated OpenGL-based scene.

## 2.1 Requirement Analysis & Design Specification

### 2.1.1 Scene Overview

The scene is an interactive living room designed on a 1300x820 pixel canvas. The room features a warm, textured wall, a gray floor, and several key furniture pieces and decorative items. The core interactive elements are a stylish floor lamp and a television set. The lamp provides a dynamic lighting effect that globally alters the scene's brightness and color saturation when toggled. The television displays a red ball that the user can move within the screen boundaries. Additional animations include the cat's tail wagging and the second hand of the wall clock rotating in real-time.

### 2.1.2 Object Components

- **Sofa:** Composed of rectangles representing the back, seat, arms, and legs.
- **Cat:** A complex object built from circles (head, eyes), triangles (ears, nose), and a dynamically rotating, bezier-curve-based tail.
- **Lamp:** Composed of a pole, base, triangular shade, and a dynamic, multi-layered circular glow effect.
- **TV Unit:** A large cabinet structure with drawers, shelves, a bookcase, and decorative items (vase, books).
- **Television:** A dark screen where a red, user-controlled ball moves.
- **Wall Clock:** A circular clock face with hour, minute, and an animated seconds hand.
- **Background:** Rectangles for the wall and floor.

## 2.2 Use of Modern Tools

This section highlights the modern software tools and technologies employed in the **Computer Graphics project** to enhance visualization, animation control, and rendering performance. It explains how these tools assist in creating dynamic 2D scenes, debugging visual elements, and improving development efficiency.

✓ **OpenGL / FreeGLUT** – Open-source graphics libraries used for rendering 2D objects, applying transformations, and implementing real-time animations.

✓ **Code::Blocks / Visual Studio** – Integrated Development Environments (IDEs) used for writing, compiling, and debugging OpenGL programs efficiently.

✓ **GLUT / FreeGLUT Libraries**– Frameworks that manage window creation, input handling, and animation timing for smoother real-time rendering.

✓ **C++ Programming Language** – Core programming language used for implementing graphics algorithms (Midpoint Circle) and modular code structure.

✓ **GIMP / Photoshop / Inkscape (Optional)** – Tools used for creating or editing textures, backgrounds, and digital assets used within the scene.

✓ **Git / GitHub** – Version control systems for managing source code, tracking changes, and collaborating effectively among team members.

## 2.3 Algorithm Used

The project implements the Midpoint Circle Algorithm for all curved objects. This algorithm was chosen for its efficiency, as it uses only integer arithmetic to determine the points of a circle, avoiding the computational overhead of floating-point or trigonometric functions. The algorithm is implemented in two forms:

- drawMidpointCircle(): Fills a circle by drawing horizontal line segments for each scan line, creating a solid disk.
- drawMidpointCircleOutline(): Draws only the circumference of the circle using the standard point-plotting method.

These functions are used to create the clock face, the lamp's radial glow, the cat's eyes and body parts, the decorative vase, and the moving ball on the TV.

## 2.4 2D Transformations Applied

- **Translation:** Applied explicitly to the moving ball on the TV screen (ballX, ballY). Also used conceptually in the animation loop to update object positions over time.
- **Rotation:** Used to animate the tail of the cat. A tailAngle variable is continuously updated and applied to calculate the position of the tail's tip using trigonometric functions (sin, cos), effectively rotating it around its base. Also used to calculate the position of the clock's second hand based on the current system time (clockSeconds).
- **Scaling:** Not explicitly used, as objects are drawn with fixed coordinates. However, the principle is demonstrated in the layered circles of the lamp's glow, which vary in radius and alpha transparency.
- **Reflection/Shear:** Not explicitly implemented in this project, but the glBegin(GL_TRIANGLES) and glBegin(GL_QUADS) primitives allow for the creation of shapes that could easily be modified with shear transformations.

## 2.5 Animation Logic

The project contains three distinct animation logics:

1. **Timer-based Animation:** A glutTimerFunc is set to call the timer() function every 16 milliseconds (~60 FPS). This function updates the tailAngle for the cat (bouncing between -38 and +38 degrees) and fetches the current system time to update the clockSeconds variable. This ensures smooth, continuous animation.
2. **Real-time User Interaction:** The specialKeys() function handles arrow key presses, which directly translate (ballX += step) the ball's position on the TV. The glutPostRedisplay() function is called to immediately redraw the scene.
3. **State-based Dynamic Lighting:** The keyboard() function toggles the global lampOn boolean variable. The roomColor() helper function checks this state and applies a darkening factor to the color of all objects before rendering, simulating a global illumination effect.

# Chapter 3

# Implementation and Results

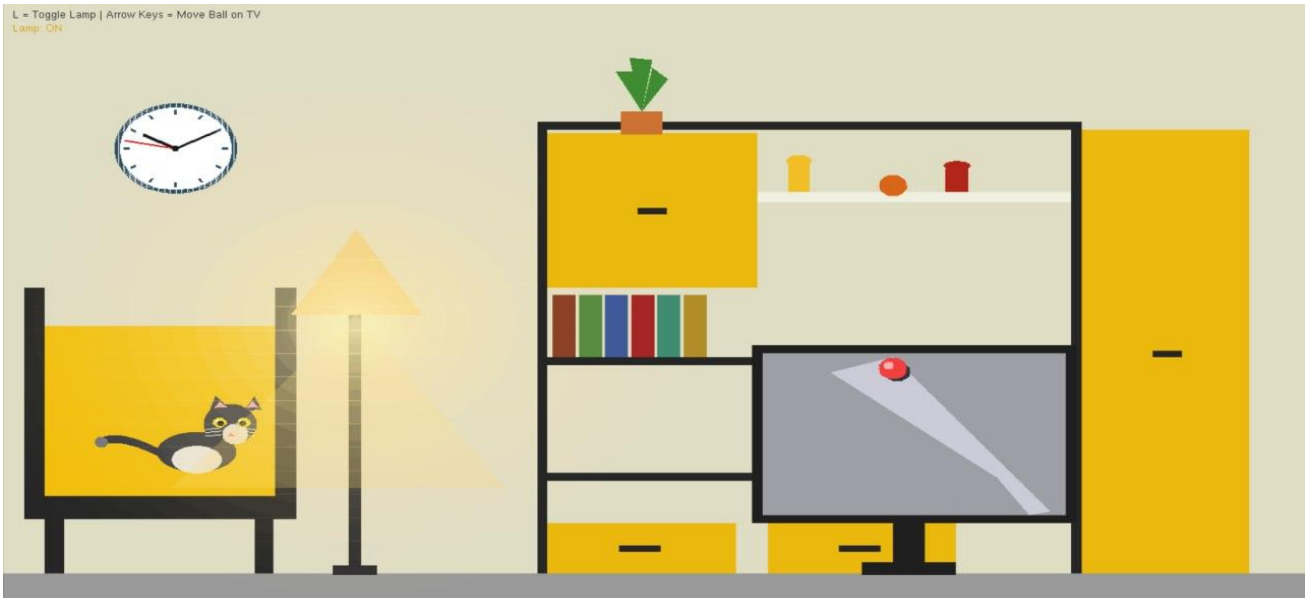
This chapter explains the step-by-step implementation process of the Computer Graphics project, including the development of objects, application of algorithms, and integration of animations. It also presents the key features, transformations, and visual outputs achieved through OpenGL rendering.

## 3.1 Implementation

The project was implemented in C++ using the OpenGL and FreeGLUT libraries. The implementation followed a modular approach:

1. **Setup and Initialization:** The main() function initializes the GLUT window, sets the display mode for double buffering and alpha blending, and defines the 2D orthographic projection. Blending is enabled for the lamp's transparent glow effect.
2. **Color Management:** A roomColor() function was created to centralize the dynamic lighting logic. It accepts RGB values and a darkening factor, applying the factor only if lampOn is false.
3. **Primitive and Algorithm Functions:** Basic drawing functions (drawRect, drawLine) and the Midpoint Circle algorithm implementations were written first. These served as building blocks for all complex objects.
4. **Object Construction:** Dedicated functions (drawSofa, drawCat, drawLamp, etc.) were created. Each function uses the basic primitives, the Midpoint Circle algorithm, and the roomColor() function to draw a specific object. The cat's tail animation uses a quadratic Bézier curve logic within a glBegin(GL_LINE_STRIP) block for a smooth, curved appearance.
5. **Animation and Interaction Logic:** The timer() function for frame-independent animations and the keyboard()/specialKeys() callbacks for user interaction were implemented.
6. **Rendering Loop:** The display() function clears the buffer, loads the identity matrix, and sequentially calls all object-drawing functions. It also renders the on-screen HUD text.

## 3.2 Output



The final output is an interactive window showing a detailed living room.

- **Visuals:** A warm-colored room contains a yellow and dark brown sofa, a cute gray cat with a pink nose and animated tail, a modern floor lamp, a large wooden TV unit, and a wall clock.
- **Dynamic Lighting:** Pressing the 'L' key toggles the lamp. When ON, a bright yellow cone and radial glow appear, and all colors are vibrant. When OFF, the entire scene darkens significantly.
- **Animations:** The cat's tail smoothly sways back and forth. The clock's second hand rotates in real-time, updating every second.
- **User Interaction:** The user can move a small red ball anywhere within the TV screen's boundaries using the arrow keys (Up, Down, Left, Right).

## 3.3 Discussion

The project successfully met all its core objectives. The use of the Midpoint Circle algorithm proved efficient for drawing multiple circular elements. The modular design made the code organized and easy to debug. The most challenging part was implementing the dynamic lighting effect, which required careful management of the OpenGL color state and alpha blending for the glow. The solution of creating a wrapper function (roomColor) elegantly handled this complexity. The real-time animations and interactions work smoothly, demonstrating a solid understanding of OpenGL's event-driven and time-based programming models. The project showcases a high level of integration, moving beyond simple primitive drawing to create a cohesive and engaging interactive application.

# Chapter 4

# Engineering Standards and Mapping

This chapter discusses the engineering standards followed throughout the project and maps the project activities to the course and program outcomes. It highlights how the project meets academic and professional requirements.

## 4.1 Impact on Society, Environment and Sustainability

### 4.1.1 Impact on Life

This project, while a simulation, has the potential to impact daily life by demonstrating how virtual environments can be used for interior design visualization, relaxation, and interactive entertainment. It provides a technical foundation for applications that allow users to visualize changes in lighting or furniture arrangement before making real-world decisions, saving time and resources.

### 4.1.2 Impact on Society & Environment

As a purely software-based project, its direct environmental impact is negligible. Indirectly, by promoting virtual prototyping and visualization, it encourages practices that can reduce material waste associated with physical mockups. The project does not contain any content that negatively impacts society; instead, it promotes creativity and technical skill development.

### 4.1.3 Ethical Aspects

In this Computer Graphics project, we follow ethical principles to ensure the design and implementation of graphical scenes are original, respectful, and free from plagiarism or unauthorized content. All visual elements, images, and animations used in the project are either self-created or properly credited to their sources, maintaining academic integrity and intellectual property rights.

Furthermore, we ensure that the project does not include any offensive, misleading, or culturally insensitive visuals. By adhering to ethical guidelines in design, documentation, and presentation, this project promotes creativity, honesty, and professionalism — essential values in responsible engineering and digital content creation.

### 4.1.4 Sustainability Plan

The project is designed to be sustainable in an academic context. The code is modular and well-commented, making it easy for future students to understand, maintain, and build upon. The use of standard, open-source libraries (OpenGL, FreeGLUT) ensures that the project does not depend on proprietary or ephemeral software.

## 4.2 Complex Engineering Problem

### 4.2.1 Mapping of Program Outcome

In this section, provide a mapping of the problem and provided solution with targeted Program Outcomes (PO’s).

Table 4.1: Justification of Program Outcomes

| POs | Justification of Mapping (Project Perspective) |
|---|---|
| PO1 | This project applies mathematical and computational knowledge by using the Midpoint Circle algorithm (integer-based circle drawing), trigonometric functions for rotation (cat's tail, clock hand), and coordinate geometry to create and transform graphical objects. |
| PO5 | The project incorporates modern computer graphics tools and technologies such as OpenGL, FreeGLUT, and C++ programming to implement a real-time, interactive application with dynamic lighting and animation, demonstrating contemporary engineering practices. |
| PO9 | Through collaborative project work, every member was engaged in teamwork, task allocation (e.g., one member focused on algorithms, another on the cat model, another on the lamp and lighting), and documentation, demonstrating effective individual and team roles. |
| PO10 | The project's outcomes are presented through this detailed technical report, a live software demonstration, and a viva voce. The report clearly communicates the technical implementation, algorithms, and results. |

### 4.2.2 Complex Problem Solving

The following table presents the mapping of the project with the **Complex Engineering Problem (CEP) attributes defined by the Washington Accord framework**. The attributes that are addressed in this project are marked (✓).

Table 4.2: Mapping with complex problem solving.

| **EP1** Dept of Knowledge | **EP2** Range of Conflicting Requirements | **EP3** Depth of Analysis | **EP4** Familiarity of Issues | **EP5** Extent of Applicable Codes | **EP6** Extent Of Stakeholder Involvement | **EP7** Inter-dependence |
|---|---|---|---|---|---|---|
| ✓ | | ✓ | ✓ | | | |

**EP1:** This project requires advanced understanding of computer graphics beyond simple drawing. It involves the efficient implementation of the Midpoint Circle algorithm, the application of trigonometric rotations for the cat's tail and clock hand, and the logical structuring of a dynamic lighting system that globally alters scene rendering. (✓)

**EP3:** The project required deep analysis of object interactions and movement. For example, the cat's tail animation was analyzed not as a simple rotation but as a curved motion, leading to the implementation of a Bézier curve. The dynamic lighting required analysis of how to apply a color transformation efficiently to every object without rewriting their drawing code. (✓)

**EP4:** The project operates within the familiar domain of computer graphics (2D shapes, OpenGL) but requires the integration of multiple techniques: geometric primitives, the Midpoint Circle algorithm, timer-based and user-input-driven transformations, and alpha blending for transparency. (✓)

### 4.2.3 Complex Engineering Activities

Table 4.3 presents the mapping of the project with the **Complex Engineering Activities (CEA) attributes**. The attributes relevant to this project are marked (✓).

Table 4.3: Mapping with complex engineering activities.

| EA1 | EA2 | EA3 | EA4 | EA5 |
|---|---|---|---|---|
| ✓ | | ✓ | | |

**EA1:** The project utilized a wide range of resources including the OpenGL/FreeGLUT API, C++ standard libraries (cmath, ctime), and online documentation for algorithm implementation. Debugging involved analyzing rendering logic, transformation sequences, and animation timing, which goes beyond routine lab tasks.

**EA3:** The project demonstrates significant creative complexity by creating a dynamic scene with multiple interacting systems. The living room is not a static collection of shapes; it features a state-based lighting system, a user-controlled object on the TV, a smoothly animated cat tail, and a real-time clock, all rendering concurrently. This required a structured graphics pipeline and careful management of the rendering loop.

# Chapter 5

# Conclusion

## 5.1 Summary

This project successfully developed an "Interactive Living Room," a dynamic 2D scene using OpenGL and C++. The project met all its objectives by implementing the Midpoint Circle algorithm, applying 2D transformations for animation, and creating a real-time interactive experience. Key features include a toggle-able dynamic lighting system, a user-controlled moving ball, an animated cat tail, and a functional wall clock. The project demonstrates a deep understanding of fundamental computer graphics concepts and their integration into a cohesive, modular, and engaging application.

## 5.2 Limitation

The project has a few limitations. The scene is strictly 2D, which limits the depth and realism of the objects. The dynamic lighting is a global color darkening effect rather than a physically accurate, per-pixel lighting model. The cat and other complex objects are composed of basic primitives, which results in a non-photorealistic, stylized appearance. Additionally, the interaction is limited to toggling the lamp and moving the ball, with no other interactive elements.

## 5.3 Future Work

Future work could involve extending the project in several ways. The scene could be converted to 3D using OpenGL's 3D projection and modeling functions. A more sophisticated lighting model could be implemented to simulate shadows and directional light from the lamp. The interactivity could be enhanced by adding more interactive objects, such as a TV that changes channels or a bookshelf from which the user can "take" books. Finally, integrating texture mapping could significantly improve the realism of the walls, floor, and furniture.

# References

[1] OpenGL Programming Guide: The Official Guide to Learning OpenGL, Addison Wesley.
[2] "OpenGL Tutorials," GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org/opengl-tutorials/.
[3] "Midpoint circle algorithm," Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/Midpoint_circle_algorithm.
[4] D. Shreiner, *OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.5 with SPIR-V*, 9th ed. Addison-Wesley Professional, 2016.